



Parul University

Man

FACULTY OF ENGINEERING AND TECHNOLOGY

BACHELOR OF TECHNOLOGY

SOFTWARE TESTING & QUALITY ASSURANCE (203105396)

6TH SEMESTER

COMPUTER SCIENCE DEPARTMENT

Laboratory Manual



TABLE OF CONTENT

Sr. No	Experiment Title	Date of Start	Date of Completion	Sign	Marks (out of 10)
1	Design test cases using Boundary value analysis.				
2	Design test cases using Equivalence class partitioning.				
3	Design independent paths by calculating cyclomatic complexity using date problem.				
4	Design test cases using Decision table Design test cases using Decision table Design independent paths by taking DD path using date problem.				
5	Understand the Automation Testing Approach (Theory Concept).				
6	Using Selenium IDE, write a test suite containing minimum 4 test cases.				
7	Install Selenium server and demonstrate it using a script in Java/PHP.				
8	Write and test a program to login a specific web page.				
9	Write and test a program to provide total number of objects present / available on the page.				
10	Write and test a program to update 10 student records into table into Excel file.				

PRACTICAL -1

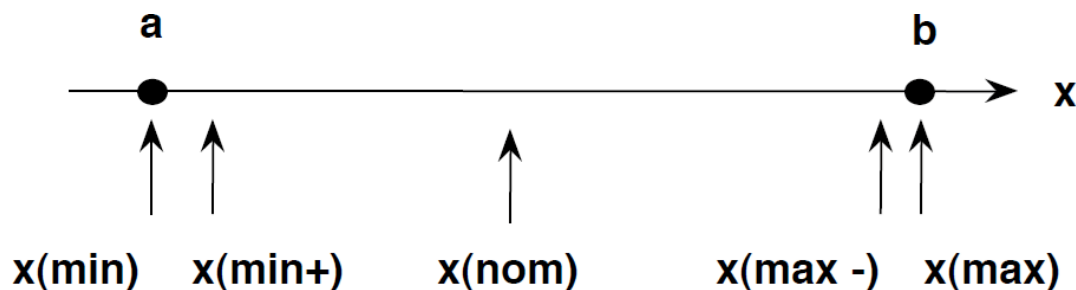
AIM: Design test cases using Boundary value analysis.

Theory:

What is Boundary Testing?

Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values.

- So these extreme ends like Start- End, Lower- Upper, Maximum-Minimum, Just Inside-Just Outside values are called boundary values and the testing is called "boundary testing".
- The basic idea in normal boundary value testing is to select input variable values at their:
 1. Minimum
 2. Just above the minimum
 3. A nominal value
 4. Just below the maximum
 5. Maximum



- In Boundary Testing, Equivalence Class Partitioning plays a good role
- Boundary Testing comes after the Equivalence Class Partitioning.

User Name:

Example on Boundary Value Analysis Test Case Design Technique:

Assume, we have to test a field which accepts User Name

Minimum boundary value is 5



Maximum boundary value is 15

Valid Inputs: 5,7,9,11,13,15

Invalid Inputs: 4 and 57

Test case	Enter the value	Valid	Invalid	Pass/fail
1	4(5-1)	No	Yes	Fail
2	9	Yes	No	Pass
3	11	Yes	No	Pass
4	15 (15+1)	Yes	No	Pass
5	56	No	Yes	Fail

PRACTICAL -2

AIM: Design test cases using Equivalence class partitioning.

Theory:

Equivalence Partitioning

Equivalence Partitioning or Equivalence Class Partitioning is type of black box testing technique which can be applied to all levels of software testing like unit, integration, system, etc. In this technique, input data units are divided into equivalent partitions that can be used to derive test cases which reduces time required for testing because of small number of test cases.

- It divides the input data of software into different equivalence data classes.
- You can apply this technique, where there is a range in the input field.

Example 1: OTP

Let us consider an example of software application. There is function of software application that accepts only particular number of digits, not even greater or less than that particular number.

Consider an OTP number that contains only 6 digit number, greater and even less than six digits will not be accepted, and the application will redirect customer or user to error page. If password entered by user is less or more than six characters, that equivalence partitioning method will show an invalid OTP. If password entered is exactly six characters, then equivalence partitioning method will show valid OTP.

Enter OTP

***Must include six digits**

Equivalence Partioning			
Invalid	Invalid	Valid	Valid
Digits>=7	Digits<=5	Digits=6	Digits=6
67545678	9754	654757	213309



PRACTICAL -3

AIM: Design independent paths by calculating cyclomatic complexity using date problem.

Theory:

Cyclomatic Complexity may be defined as:

- It is a software metric that measures the logical complexity of the program code.
- It counts the number of decisions in the given program code.
- It measures the number of linearly independent paths through the program code.

Cyclomatic complexity indicates several information about the program code

Cyclomatic Complexity	Meaning
1 – 10	• Structured and Well Written Code • High Testability • Less Cost and Effort
10 – 20	• Complex Code • Medium Testability • Medium Cost and Effort
20 – 40	• Very Complex Code • Low Testability • High Cost and Effort
> 40	• Highly Complex Code • Not at all Testable • Very High Cost and Effort

Importance of Cyclomatic Complexity-

- It helps in determining the software quality.



- It is an important indicator of program code's readability, maintainability and portability.
- It helps the developers and testers to determine independent path executions.
- It helps to focus more on the uncovered paths.
- It evaluates the risk associated with the application or program.
- It provides assurance to the developers that all the paths have been tested at least once.

Properties of Cyclomatic Complexity-

- It is the maximum number of independent paths through the program code.
- It depends only on the number of decisions in the program code.
- Insertion or deletion of functional statements from the code does not affect its cyclomatic complexity.
- It is always greater than or equal to 1.

$$\text{Cyclomatic Complexity} = E - N + 2$$

Here-

- E = Total number of edges in the control flow graph
- N = Total number of nodes in the control flow graph

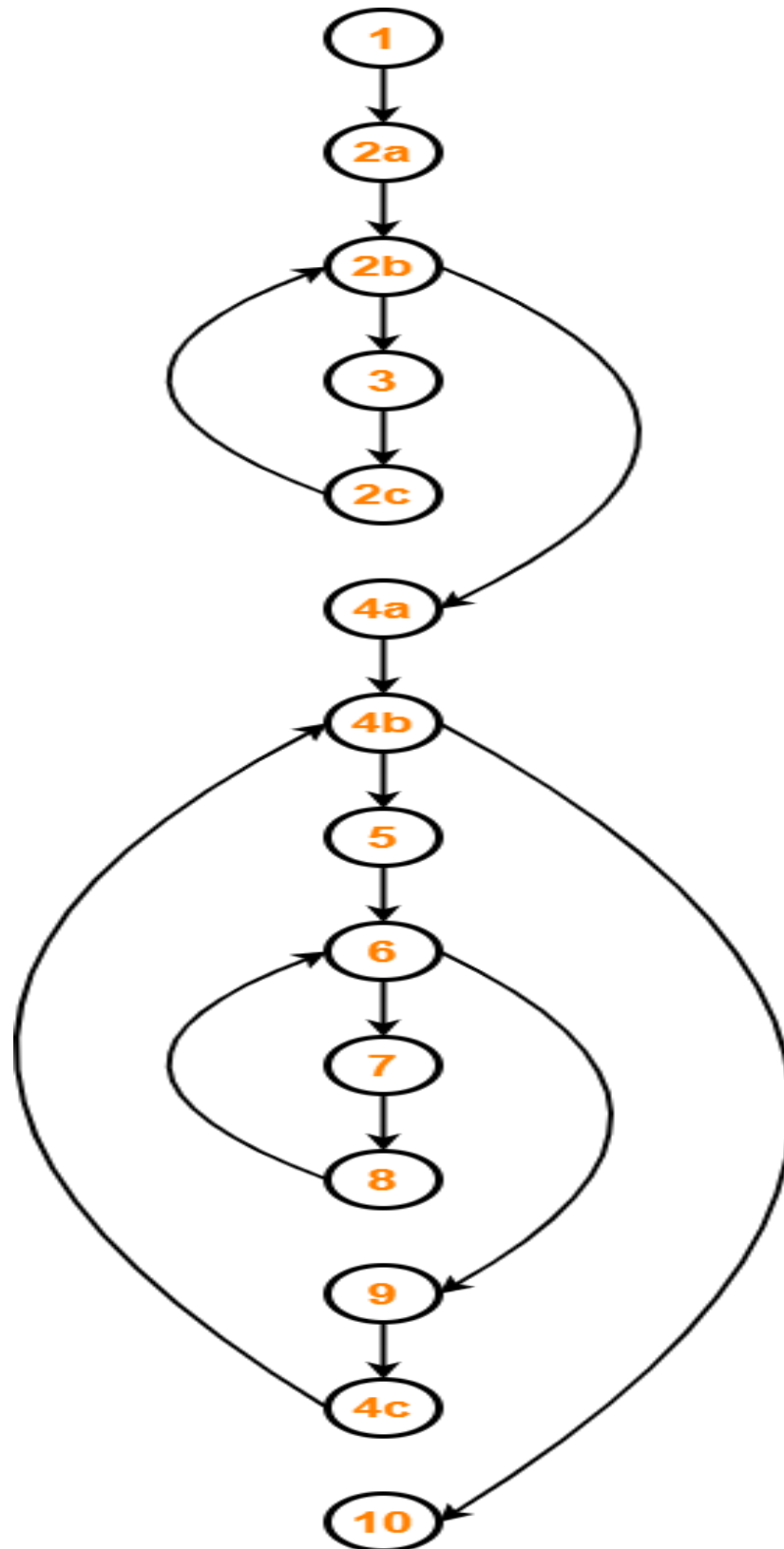
Example 1: Calculate cyclomatic complexity for the given code

```
{int i,j,k;  
For(i=0;i<=N;i++)  
P[i]=1;  
For(i=2;i<=N;i++)  
{  
K=p[i];j=1;  
While(a[p[j-1]]>a[k]){  
P[j]=p[j-1];  
j--;  
}  
P[j]=k;  
}}
```



Solution

We draw the following control flow graph for the given code



Control Flow Graph



Cyclomatic Complexity

$$= E - N + 2$$

$$= 16 - 14 + 2$$

$$= 4$$



PRACTICAL -4

AIM: Design test cases using Decision table Design independent paths by taking DD path using date problem.

Theory:

A **Decision Table** is a tabular representation of inputs versus rules/cases/test conditions. It is a very effective tool used for both complex software testing and requirements management. Decision table helps to check all possible combinations of conditions for testing and testers can also identify missed conditions easily. The conditions are indicated as True(T) and False(F) values.

Decision table testing is a software testing technique used to test system behavior for different input combinations. This is a systematic approach where the different input combinations and their corresponding system behavior (Output) are captured in a tabular form. That is why it is also called as a **Cause-Effect** table where Cause and effects are captured for better test coverage.

Let's learn with an example.

Example 1: Credit card example:

Let's take another example. If you are a new customer and you want to open a credit card account then there are three conditions first you will get a 15% discount on all your purchases today, second if you are an existing customer and you hold a loyalty card, you get a 10% discount and third if you have a coupon, you can get 20% off today (but it can't be used with the 'new customer' discount).

Discount amounts are added, if applicable. This is shown in Table

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8
New customer(15%)	T	T	T	T	F	F	F	F
Loyalty card(10%)	T	T	F	F	T	T	F	F
Coupon(20%)	T	F	T	F	T	F	T	F
Actions								
Discount(%)	X	X	20	15	30	10	20	0

the conditions and actions are listed in the left hand column. All the other columns in the decision table each represent a separate rule, one for each combination of conditions.

We may choose to test each rule/combination and if there are only a few this will usually be the case. However, if the number of rules/combinations is large we are more likely to sample them by selecting a rich subset for testing.

Now let's see the decision table for credit card shown above:

- Note that we have put X for the discount for two of the columns (Rules 1 and 2) – this means that this combination should not occur. You cannot be both a new customer and also holding a loyalty card as per the conditions mentioned above. Hence there should be an error message stating this.
- We have made an assumption in Rule 3. Since the coupon has a greater discount than the new customer discount, we assume that the customer will choose 20% rather than 15%. We cannot add them, since the coupon cannot be used with the 'new customer' discount as stated in the condition above. The 20% action is an assumption on our part, and we should check that this assumption (and any other assumptions that we make) is correct, by asking the person who wrote the specification or the users.
- For Rule 5, however, we can add the discounts; since both the coupon and the loyalty card discount should apply (that's our assumption).
- Rules 4, 6 and 7 have only one type of discount and Rule 8 has no discount, so 0%.

If we are applying this technique thoroughly, we would have one test for each column or rule of our decision table. The advantage of doing this is that we may test a combination of things that otherwise we might not have tested and that could find a defect.

However, if we have a lot of combinations, it may not be possible or sensible to test every combination. If we are time-constrained, we may not have time to test all combinations. Don't just assume that all combinations need to be tested.

It is always better to prioritize and test the most important combinations. Having the full table helps us to decide which combinations we should test and which not to test this time.

In the example above all the conditions are binary, i.e. they have only two possible values: True or False (or we can say Yes or No).

Design independent paths by taking DD path using date problem

To draw a Flow Graph, a DD Graph, calculation of Cyclomatic Complexity $V(G)$ and find out all independent paths from the DD paths graph, for the case of a triangle wherein the program reads the three sides of a triangle (say a, b, c). The output may be Scalene triangle or a Isosceles triangle or an equilateral triangle or might not be a triangle at all.



When we have a flow graph, we can easily draw another graph that is known as decision-to-decision or (DD) path graph, wherein we lay our main focus on the decision nodes only. The nodes of the flow graph are combined into a single node if they are in sequence.



PRACTICAL -5

AIM: Understand the Automation Testing Approach (Theory Concept).

Theory:

Automation Testing or Test Automation is a software testing technique that performs using special automated testing software tools to execute a test case suite. On the contrary, Manual Testing is performed by a human sitting in front of a computer carefully executing the test steps.

The automation testing software can also enter test data into the System under Test, compare expected and actual results and generate detailed test reports. Software Test Automation demands considerable investments of money and resources.

Successive development cycles will require execution of same test suite repeatedly. Using a test automation tool, it's possible to record this test suite and re-play it as required. Once the test suite is automated, no human intervention is required. This improved ROI of Test Automation. The goal of Automation is to reduce the number of test cases to be run manually and not to eliminate Manual testing altogether.

Test Automation is the best way to increase the effectiveness, test coverage, and execution speed in software testing. Automated software testing is important due to the following reasons:

- Manual Testing of all workflows, all fields, all negative scenarios is time and money consuming
- It is difficult to test for multilingual sites manually
- Test Automation in software testing does not require Human intervention. You can run automated test unattended (overnight)
- Test Automation increases the speed of test execution
- Automation helps increase Test Coverage
- Manual Testing can become boring and hence error-prone.



Which Test Cases to Automate?

Test cases to be automated can be selected using the following criterion to increase the automation ROI

- High Risk - Business Critical test cases
- Test cases that are repeatedly executed
- Test Cases that are very tedious or difficult to perform manually
- Test Cases which are time-consuming

The following category of test cases is not suitable for automation:

- Test Cases that are newly designed and not executed manually at least once
- Test Cases for which the requirements are frequently changing
- Test cases which are executed on an ad-hoc basis.

Automated Testing Process:

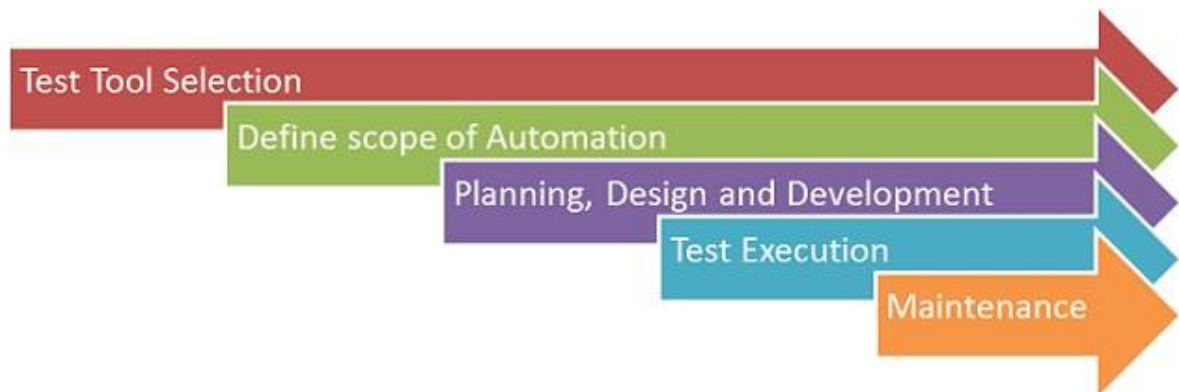
Step 1) Test Tool Selection

Step 2) Define scope of Automation

Step 3) Planning, Design and Development

Step 4) Test Execution

Step 5) Maintenance



Test tool selection

Test Tool selection largely depends on the technology the Application Under Test is built on. For instance, QTP does not support Informatica. So QTP cannot be used for testing Informatica applications. **It's a good idea to conduct a Proof of Concept of Tool on AUT.**

Define the scope of Automation

The scope of automation is the area of your Application Under Test which will be automated. Following points help determine scope:

- The features that are important for the business
- Scenarios which have **a large amount of data**
- **Common functionalities** across applications
- Technical feasibility
- The extent to which business components are reused
- **The complexity** of test cases
- Ability to use the same test cases for cross-browser testing

TestProject

TestProject is the world's first free cloud-based, community-powered test automation platform that enables users to test Web, Android and iOS applications on all operating systems, effortlessly. Easily collaborate with your team using Selenium and Appium to ensure quality with speed. Use advanced built-in recording capabilities, create and use addons (automation actions shared by the entire community), or develop coded tests using TestProject's powerful SDK, all completely for FREE!

Features:



- No complex setups or configurations
- No coding skills required to get started
- Share and reuse addons with your team and the entire community
- Detailed report dashboards
- Seamless integrations with your CI/CD workflow



PRACTICAL -6

AIM: Using Selenium IDE, write a test suite containing minimum 4 test cases.

Theory:

Selenium IDE TEST suite

<https://procapital.mohdfaiz.com/calculator.php>

The screenshot displays the Selenium IDE interface within a Mozilla Firefox browser window. The project is named 'CAGR Calculator'. The URL bar shows 'https://procapital.mohdfaiz.com/calculator.php'. The test suite 'Jay Depani 1*' is selected, showing a table of commands:

Command	Target	Value
type	id=Arate	15
click	id=tenure	
type	id=tenure	25
click	id=submit	

Below the table, there are input fields for Command, Target, Value, and Description. The bottom section shows a log of test execution steps, including clicking on frequency, selecting frequency, clicking on investment type, clicking on Minvestment, typing investment value, clicking on Arate, typing Arate value, clicking on tenure, typing tenure value, and clicking on submit. The final status is 'Jay Depani 1* completed successfully'.



Selenium IDE - CAGR Calculator* - Mozilla Firefox

Project: CAGR Calculator*

Tests + [Run] [Stop] [Refresh] [Reset]

Search tests... Run current test Ctrl+R aiz.com/calculator.php

	Command	Target	Value
Jay Depani 1*			
Jay Depani 2*	7 type	id=Arate	15
	8 click	id=tenure	
	9 type	id=tenure	25
	10 click	id=submit	

Command [] # [] []
Target [] [] []
Value []
Description []

Log Reference

1. open on https://procapital.mohdfaiz.com/calculator.php OK 17:59:04
2. setWindowSize on 1452x805 OK 17:59:04
3. click on css=col-md-6:nth-child(4) article-image OK 17:59:04
4. click on id=Minvestment OK 17:59:09
5. type on id=Minvestment with value 10000 OK 17:59:10
6. click on id=Arate OK 17:59:11
7. type on id=Arate with value 15 OK 17:59:11
8. click on id=tenure OK 17:59:11
9. type on id=tenure with value 25 OK 17:59:11
10. click on id=submit OK 17:59:11

'Jay Depani 2' completed successfully 17:59:11

Selenium IDE - CAGR Calculator* - Mozilla Firefox

Project: CAGR Calculator*

Tests + [Run] [Stop] [Refresh] [Reset]

Search tests... Run current test Ctrl+R aiz.com/calculator.php

	Command	Target	Value
Jay Depani 1*	5 type	id=Investment	10000
Jay Depani 2*	6 click	id=Eninvestment	
Jay Depani 3*	7 type	id=Eninvestment	2000000000
	8 click	id=tenure	
	9 type	id=tenure	25
	10 click	id=submit	

Command [] # [] []
Target [] [] []
Value []
Description []

Log Reference

1. open on https://procapital.mohdfaiz.com/calculator.php OK 18:00:32
2. setWindowSize on 1452x805 OK 18:00:33
3. click on css=col-md-6:nth-child(5) article-image OK 18:00:33
4. click on id=Investment OK 18:00:37
5. type on id=Investment with value 10000 OK 18:00:41
6. click on id=Eninvestment OK 18:00:41
7. type on id=Eninvestment with value 2000000000 OK 18:00:41
8. click on id=tenure OK 18:00:41
9. type on id=tenure with value 25 OK 18:00:41
10. click on id=submit OK 18:00:42

'Jay Depani 3' completed successfully 18:00:42



Selenium IDE - CAGR Calculator* - Mozilla Firefox

Project: CAGR Calculator*

Tests + [Run] [Stop] [Refresh] [Reset]

Search tests... Run current test Ctrl+R aiz.com/calculator.php

	Command	Target	Value
Jay Depani 1*	5 click	id=Eninvestment	
Jay Depani 2*	6 type	id=Eninvestment	15
Jay Depani 3*	7 click	id=tenure	
Jay Depani 4*	8 type	id=tenure	25
	9 click	id=submit	
	10 click	id=Eninvestment	

Command: [] [] [] []
Target: [] [] [] []
Value: []
Description: []

Log Reference

1. open on https://procapital.mohdfaiz.com/calculator.php OK 18:03:07
2. setWindowSize on 1452x805 OK 18:03:07
3. runScript on window.scrollTo(0,479.20001220703125) OK 18:03:07
4. click on css= col-md-6:nth-child(6) .article-image OK 18:03:11
5. click on id=Eninvestment OK 18:03:11
6. type on id=Eninvestment with value 15 OK 18:03:14
7. click on id=tenure OK 18:03:15
8. type on id=tenure with value 25 OK 18:03:15
9. click on id=submit OK 18:03:15
10. click on id=Eninvestment OK 18:03:15

'Jay Depani 4' completed successfully 18:03:15



PRACTICAL -7

Aim: Install Selenium server and demonstrate it using a script in Java/PHP.

We need to download Selenium RC server / client driver and configure that to Eclipse

1. Download Selenium server: <http://seleniumhq.org/download/>
2. Download Selenium Client driver for Java (from Selenium ClientDrivers section)
3. Create "Selenium" folder in C: drive and copy the Selenium-server.jar as well as unzip the Selenium Client driver(C:Selenium)

Downloading and unzipping the files into a folder is done. We need to configure the appropriate Selenium Client driver Jar file to the Eclipse.

1. Go to Eclipse → Click File → New → Project (from various options need to select just "project")
2. In Select Wizard → Click Java → "Java Project" (demonstrated in the below figure)
3. Give the project name (e.g. SugarCRMTTests)
4. Click Finish – Click Yes
5. Now we are done with creation of project and need to configure the Selenium Client driver to this Project
6. Right Click "SugarCRMTTests" project
7. Click "Java Build Path"
8. Click Libraries tab
9. Click "Add External JARs" button
10. Select "Selenium Client Drivers" unzipped in C:Selenium folder (Selenium Server JAR file should not be added)
11. Click OK
12. Referenced libraries → contains both the Selenium Client driver jar files as shown in the below picture.

Extending the Selenese Test Case

1. Install any IDE (Eclipse, IntelliJ, IDEA), create a project.
2. Download the selenium RC file from [Selenium website](http://seleniumhq.org/)
3. Extract the selenium-remote-control-1.0.3.zip file and add the selenium-remote-control-1.0.3\selenium-java-client-driver-1.0.1\selenium-java-clientdriver.jar and selenium-remote-control-1.0.3\selenium-server-1.0.3\selenium-server.jar file to the build path.
4. Create a new package and a class in the newly created package.
5. Copy the below code:



```
import com.thoughtworks.selenium.SeleneseTestCase;
public class SeleniumTest1 extends SeleneseTestCase {
    public void setUp() throws Exception {
        setUp("http://www.hexbytes.com/", "*firefox");
    }
    public void testNew() throws Exception{
        selenium.open("");
        selenium.type("searchbox", "selenium rc");
        selenium.click("css=input[type='submit'][value='Search']");
        selenium.waitForPageToLoad("30000");
        assertTrue(selenium.isTextPresent("selenium rc"));
    }
}
```

6. Start the selenium server.
7. Click on the Run As button and run the script as JUnit Test. If you don't find the option Install the JUnit plug-in and run the script.



PRACTICAL -8

Aim: Write and test a program to login a specific webpage.

```
import com.thoughtworks.selenium.*;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import java.util.regex.Pattern;

public class exp5 extends SeleneseTestCase
{
    @Before
    public void setUp() throws Exception {
        selenium = new DefaultSelenium("localhost", 4444, "*chrome",
"http://demo.opensourcecms.com/");
        selenium.start();
    }

    @Test
    public void testExp5() throws Exception {
        selenium.open("/wordpress/wp-login.php");
        selenium.type("id=user_login", "admin");
        selenium.type("id=user_pass", "demo123");
        selenium.click("id=wp-submit");
        selenium.waitForPageToLoad("30000");
    }

    @After
    public void tearDown() throws Exception {
        selenium.stop();
    }
}
```

TestNG

TestNG is a testing framework inspired from JUnit and NUnit but introducing some new functionalities that make it more powerful and easier to use, such as:

- Annotations.
- Run your tests in arbitrarily big thread pools with various policies available (all methods in their own thread, one thread per test class, etc...).

- Test that your code is multithreadsafe.
- Flexible testconfiguration.
- Support for data-driven testing (with@DataProvider).
- Support for parameters.
- Powerful execution model (no moreTestSuite).
- Supported by a variety of tools and plug-ins (Eclipse, IDEA, Maven,etc...).
- Embeds BeanShell for furtherflexibility.
- Default JDK functions for runtime and logging (nodependencies).
- Dependent methods for application servertesting.

TestNG is designed to cover all categories of tests: unit, functional, end-to-end, integration, etc...

Installing TestNG in eclipse

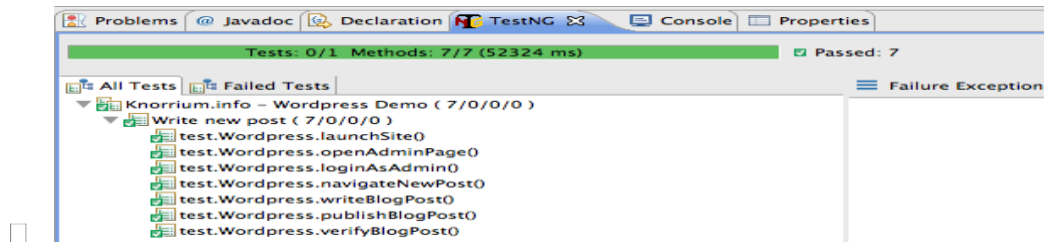
- Select Help / Software updates / Find andInstall.
- *Search for new features toinstall.*
- New remotesite.
- For Eclipse 3.4 and above, enter<http://beust.com/eclipse>.
- For Eclipse 3.3 and below, enter<http://beust.com/eclipse1>.
- Make sure the check box next to URL is checked and clickNext.
- Eclipse will then guide you through theprocess.

Launching your tests in Eclipse

- We finished writing our tests, now how can we runthem?
- You can launch TestNG from the command line, using a Eclipse pluginor even programatically. We are going to use the Eclipse plugin. Follow the steps described on the official TestNG documentation[overhere](#)
- If you installed TestNG correctly, you will see this menu when youright click on the XMLfile:



- Click on “Run as TestNG Suite” and your test will start running. You will then see this nice result tree:



Selenium Tests with Microsoft Excel

Parameterizing a test from external sources such as Microsoft Excel is always recommended in order to handle large amount of test data. To read data from Excel, we need APIs which support opening file, reading data, and writing data into Excel. We should know various classes and methods which support above mentioned operations. In this post, let us try to figure out which is the API that supports all the activities we need to do during execution of a test.

Jxl.jar is an open source Java API which supports read Excel spreadsheets and to write into Excel spreadsheets. Below are some of the operations that we can handle with this API.

1. Read data from Excel spreadsheet
2. Read and write formulas into spreadsheets
3. Generate spreadsheets
4. Supports formatting of font, number, and date
5. Supports coloring of cells

To access the methods and classes provided by this API inside Eclipse we need to add this JAR file to the Java Build Path. (I have explained steps to add external Jar files to Java Build Path in previous posts)

Download the jxl.jar from “<http://jexcelapi.sourceforge.net/>”

Add the JAR file to Java Build Path

Add **import** statements to the .java file as below to read from an Excel spreadsheet

import jxl.Cell;



```
import jxl.Sheet;
```

```
import jxl.Workbook;
```

```
import jxl.read.biff.BiffException;
```



PRACTICAL -9

Aim: Write and test a program to provide total number of objects present / available on the page.

```
package testscripts;

import com.thoughtworks.selenium.*;
import org.openqa.selenium.server.*;
import org.testng.annotations.*;

public class exp8 {

    public Selenium selenium;

    public SeleniumServer seleniumserver;

    @BeforeClass
    public void setUp() throws Exception {

        RemoteControlConfiguration rc = new RemoteControlConfiguration();
        seleniumserver = new SeleniumServer(rc);
        selenium = new DefaultSelenium("localhost", 4444, "firefox", "http://");
        seleniumserver.start();
        selenium.start();
    }

    @Test
    public void testDefaultTNG() throws Exception {

        selenium.open("http://www.google.co.in/");
        selenium.windowMaximize();
        String lc[]=selenium.getAllLinks();
        System.out.println("TOTAL NO OF LINKS="+lc.length);

        String bc[]=selenium.getAllButtons(); System.out.println("TOTAL
        NO OF BUTTONS="+bc.length); String
        fc[]=selenium.getAllFields();
        System.out.println("TOTAL NO OF INPUT FIELDS="+fc.length);
    }

    @AfterClass
```



```
public void tearDown() throws Exception {  
    selenium.stop();  
  
    seleniumserver.stop();  
  
}  
}
```

OUTPUT

TOTAL NO OF LINKS=41 TOTAL
NO OF BUTTONS=1 TOTAL NO
OF INPUT FIELDS=3



PRACTICAL -10

Aim: Write and test a program to update 10 student records into table into Excel file.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import jxl.Sheet;

import jxl.Workbook;
import jxl.write.Label;

import jxl.write.WritableSheet;
import jxl.write.WritableWorkbook;
import org.testng.annotations.*;
public class newj {

    @BeforeClass
    public void setUp() throws Exception {}
    @Test

    public void testImportexport1() throws Exception {
        FileInputStream fi = new FileInputStream("D:\\exp6.xls");
        Workbook w = Workbook.getWorkbook(fi);

        Sheet s = w.getSheet(0);
        String a[][] = new String[s.getRows()][s.getColumns()];
        FileOutputStream fo = new FileOutputStream("D:\\exp6Result.xls");
        WritableWorkbook ww = Workbook.createWorkbook(fo);
        WritableSheet ws = ww.createSheet("result1", 0);
        for (int i = 0; i < s.getRows(); i++)
            for (int j = 0; j < s.getColumns(); j++)
            {

                a[i][j] = s.getCell(j, i).getContents();
                Label l2 = new Label(j, i, a[i][j]);
                ws.addCell(l2);
                Label l1 = new Label(6, 0,
                "Result"); ws.addCell(l1);
            }
        for (int i = 1; i < s.getRows(); i++) {

            for (int j = 2; j < s.getColumns(); j++)
            {
```



```
a[i][j] = s.getCell(j,i).getContents();
intx=Integer.parseInt(a[i][j]);
if(x > 35)
{
    Label l1 = new Label(6, i,
    "pass"); ws.addCell(l1);
}
else
{
    Label l1 = new Label(6, i, "fail");
    ws.addCell(l1);
    break;
}
}
}
wwb.write();
wwb.close();
}
```

Input:

rollno	Names	BMSIC	JAVA	WP	total
1	ganesh	62	85	64	211
2	praveen	62	62	64	188
3	pandurang	30	36	40	106
4	rohan	50	55	25	130
5	abc	45	20	49	114
6	qwqew	90	75	89	254

Output:

rollno	Names	BMSIC	JAVA	WP	total	Result
1	ganesh	62	85	64	211	pass
2	praveen	62	62	64	188	pass
3	pandurang	30	36	40	106	fail
4	rohan	50	55	25	130	fail
5	abc	45	20	49	114	fail
6	qwqew	90	75	89	254	pass